

CreationsForge Engineering Review

Generated for repository C:/Repositories/Personal/CreationsForge. Review scope included application code, tests, project files, packaging/configuration assets visible in the repository, database migrations/repositories, logging, file/archive parsing, UI workflows, and dependency manifests. No application source files were changed.

A. Executive Summary

Overall health is solid for a prototype moving toward production shape. The codebase has clear project boundaries, named parameterized SQL, defensive path checks for asset resolution, bounded preview asset reads, explicit database reset guardrails, and meaningful unit/headless coverage. Restore, build, unit tests, presentation tests, and package vulnerability audit all passed in this environment.

No critical security issue was confirmed from source inspection. The most important risk is data correctness during partial record imports: a changed plugin can commit newly imported rows while retaining stale rows for record types that had failures. The next priorities are limiting UI/database materialization in plugin search and record/asset browsing paths, enforcing the repository's analyzer policy in build configuration, and tightening lifecycle cleanup for cancellation sources and configuration IO failure handling.

Validation summary: dotnet restore passed; dotnet build --no-restore passed with 0 warnings and 0 errors; unit tests passed 364/364; presentation tests passed 50/50; dotnet list package --vulnerable --include-transitive reported no vulnerable packages after network approval.

Architecture And Trust Boundary Map

- Entry points: CreationsForge.Program/App for Avalonia UI and CreationsForge.Console/Program.cs for CLI import.
- Composition: CreationsForge.Bootstrap registers Core, Migrations, Starfield, Fallout4, and Skyrim Autofac modules.
- Persistence: Core owns NPoco repositories and SQLite connection setup; Migrations owns DbUp SQL scripts and SchemaVersions.
- External integrations: Mutagen reads game/plugin data inside game adapter projects; asset preview reads local loose files, BA2/BSA archives, NIF/mesh/material bytes, and may launch NifSkopec or shell file associations on Windows.
- Trust boundaries: configured application/database/log paths, installed game data folders, plugin/archive/NIF/material files, persisted plugin diagnostics, and optional NifSkopec executable path.
- Important positive controls: named SQL parameters, table-name allowlisting for dynamic record repositories, SQLite foreign keys, reset target validation, asset path containment checks, archive range validation, preview byte limits, and headless presentation tests.

B. Critical Findings

No critical findings were confirmed. This review did not identify hardcoded secrets, obvious SQL injection in runtime SQL, unsafe command argument concatenation, or unbounded archive extraction without a preview byte limit. Human review is still needed for deployment/runtime assumptions that are not visible in the repository, especially file permissions on application data directories and production log retention/access.

C. High-Priority Findings

H1. Partial record imports can leave stale rows mixed with newly imported data

Severity	High	Category	Reliability / Data Integrity
Location	CreationsForge.Core/Services/RecordImporterService.cs:141-148; CreationsForge.Core/Importers/GameImporter.cs:110-113, 314-324	Validation	Add a GameImporter/RecordImportService test where a changed plugin has one importer failure and assert stale rows are not exposed as current; run unit tests and a manual import of a fixture plugin with an injected mapper failure.

Problem: RecordImportService skips stale cleanup for a record type when any record in that type fails, but GameImporter still completes the surrounding transaction and marks the plugin PartiallyImported.

Risk: After a changed plugin is reimported, old rows for failed record types can remain beside newly imported rows. UI comparison and record browsing may present stale data as if it belongs to the latest plugin source, which is especially risky for record-diff decisions.

Evidence: Failures increment RecordsFailed at RecordImportService.cs:141, stale deletion runs only when RecordsFailed == 0 at 146-148, and GamelImporter completes the transaction at 113 before saving partial import state at 314-324.

Recommendation: For failed record types, either roll back that record type/import batch, isolate the new batch until it succeeds, or explicitly mark/read rows by import batch so stale rows are excluded from normal browsing/comparison for PartiallyImported plugins.

H2. Plugin search materializes every match and only limits after returning to the view model

Severity	High	Category	Performance
Location	CreationsForge.Core/Repositories/PluginRepository.cs:89-106; CreationsForge/ViewModels/MainViewModel.cs:569-573	Validation	Add repository/service tests for limited suggestions and exact selection; manually test typing in Open Plugin with a large imported plugin set.

Problem: SearchOpenablePluginsByFilename uses LIKE '%term%' and returns all matching rows; MainViewModel then applies Take(25) after the database result has already been materialized.

Risk: Large imported load orders or broad searches can trigger unnecessary database scans, object allocation, and UI latency on every search-text update. The pattern also makes later growth harder if plugin counts or indexed metadata increase.

Evidence: PluginRepository builds searchPattern = \$"{searchFilename}%" and Fetches all matches, then MainViewModel stores MatchingPlugins and takes only 25 for display.

Recommendation: Push a LIMIT into the repository query for suggestion use, add an index-friendly normalized/search column if needed, and keep a separate exact lookup path for submitting a plugin filename.

D. Medium-Priority Findings

M1. Build configuration does not enforce the repository's analyzer/warnings-as-errors policy

Severity	Medium	Category	Best Practice / Maintainability
Location	CreationsForge.Core/CreationsForge.Core.csproj:3-7; CreationsForge/CreationForge.csproj:3-17; solution-wide rg found no TreatWarningsAsErrors or analyzer settings	Validation	Run dotnet build from a clean checkout and confirm a deliberately introduced warning fails the build; include this in CI if applicable.

Problem: Project files enable nullable but do not visibly set TreatWarningsAsErrors, WarningsAsErrors, analysis mode, or a shared analyzer policy.

Risk: The current build has zero warnings, but future analyzer warnings may not fail CI/local validation despite the repo rule that analyzer warnings are treated as errors.

Evidence: A solution-wide search for TreatWarningsAsErrors, WarningsAsErrors, AnalysisMode, EnableNETAnalyzers, and related settings returned no matches; csproj files show Nullable enabled but no equivalent hardening.

Recommendation: Add a Directory.Build.props with the intended analyzer policy, or explicitly document why this repo enforces warnings elsewhere. Consider enabling latest recommended analyzers and treating nullable/analyzer warnings as errors.

M2. CancellationTokenSource instances are canceled but not disposed in presentation view models/views

Severity	Medium	Category	Reliability / Resource Management
----------	--------	----------	-----------------------------------

Location	CreationsForge/ViewModels/AssetPreviewPaneViewModel.cs:29, 339-363; CreationsForge/ViewModels/ImportProgressViewModel.cs:19	Validation	Add presentation tests that trigger repeated preview loads/cancel paths and verify a fake disposable source/lifetime wrapper is disposed; run presentation tests.
----------	---	------------	---

Problem: Asset preview and import view models own CancellationTokenSource instances but do not consistently dispose them after cancellation/replacement or when the view model lifetime ends.

Risk: The leak is small per source, but preview selection can churn sources repeatedly and import screens may be recreated. This is a classic managed-resource lifecycle gap and can hide wait handles/timers in future changes.

Evidence: ResetPreviewLoadCancellationTokenSource cancels the previous source at line 349 but does not dispose it; CancelCurrentPreviewLoad cancels at line 362 but does not dispose. ImportProgressViewModel holds a readonly source for its lifetime.

Recommendation: Dispose replaced sources after canceling, make owning view models IDisposable when they hold CTS instances, and ensure navigation/lifetime scopes dispose them. Keep disposal off paths where callbacks might still use the source.

M3. Configuration and logging path IO failures are not handled as gracefully as malformed JSON

Severity	Medium	Category	Reliability
Location	CreationsForge.Core/Configuration/ApplicationConfigurationStore.cs:76, 88, 103; CreationsForge.Bootstrap/Logging/SerilogConfigurator.cs:14	Validation	Add tests for unreadable/missing-parent/invalid path scenarios using temporary directories and mocked or permission-controlled files; manually verify startup with an invalid configured log directory.

Problem: Configuration load recovers from invalid JSON, but File.ReadAllText, File.WriteAllText, and logging-directory creation can still throw IO/Unauthorized/Path exceptions during startup or settings save.

Risk: A bad custom path, permission change, locked file, or corrupt filesystem state can prevent the UI/CLI from starting or saving configuration without a targeted recovery message.

Evidence: Only JsonException is caught at ApplicationConfigurationStore.cs:88. SerilogConfigurator creates the configured logging directory before logger setup at line 14.

Recommendation: Handle expected IO exceptions with clear fallback behavior: load defaults when reads fail, report save failures to the caller/UI, and fall back to a safe default log path if configured logging directory creation fails.

M4. Full-path and exception diagnostics may expose local environment details through persisted plugin state and UI

Severity	Medium	Category	Security / Privacy
Location	CreationsForge.Core/Importers/GameImporter.cs:284, 323-324; CreationsForge/ViewModels/OpenPluginRowViewModel.cs:93-95; Serilog path logging across import/asset services	Validation	Add tests around failed import state mapping to assert user-facing ImportDetails excludes stack traces/absolute profile paths; manually review generated logs for path redaction expectations.

Problem: Metadata import failures persist ex.ToString() into Plugin.ImportDetails, and the Open Plugin UI surfaces ImportDetails directly. Logs also include full archive, database, temp, and asset paths.

Risk: This is a local desktop app, so exposure is not automatically critical. However, reports/screenshots/database copies/log bundles can leak usernames, installed-game paths, machine layout, and stack traces.

Evidence: GameImporter stores ex.ToString() at line 284 and saves failed-record summaries in ImportDetails at 323-324; OpenPluginRowViewModel returns ImportDetails for display.

Recommendation: Store concise sanitized user-facing diagnostics in the database and keep full exception details in protected logs. Consider redacting user profile roots in UI-visible text and support-bundle outputs.

M5. Asset/archive lookup can still materialize large archive directories and candidate sets

Severity	Medium	Category	Performance
Location	CreationsForge.Bethesda.Assets/Archives/Ba2/Ba2ArchiveReader.cs:11, 38-40, 121-126; CreationsForge.Bethesda.Assets/Archives/Bsa/BsaArchiveReader.cs:11, 20-22, 124-129; CreationsForge.Core/Services/AssetArchiveIndexService.cs:105, 200, 486	Validation	Benchmark indexing and preview lookup against large real archives; add stress tests with high entry-count synthetic archives and assert memory stays within an expected envelope.

Problem: Entry byte reads are bounded, but archive directory/name tables are materialized and cached per archive, and lazy lookup can enumerate/archive-index multiple candidates.

Risk: Large BA2/BSA archives can contain many entries. Holding up to eight full directories per reader and repeated ToList materialization can create memory pressure during import or preview, especially across all-game workflows.

Evidence: Both archive readers cache up to eight directories. AssetArchiveIndexService materializes archive lists at lines 105 and 200 and enumerates data-folder archives at 486.

Recommendation: Track entry-count and memory metrics during real imports; consider lower cache sizes, scoped cache clearing around preview operations, streaming index writes, and explicit maximum entry-count guards for malformed archives.

E. Low-Priority Cleanup

L1. External asset opening trusts any configured existing .exe as the NifSkopec launcher

Severity	Low	Category	Potential Security
Location	CreationsForge/Services/ExternalAssetPathPolicy.cs:18-21; CreationsForge/Services/ExternalAssetOpenService.cs:126-129, 160-163	Validation	Manual review of application-data permissions; add tests for changed executable path prompting if implemented.

Problem: The policy checks that the configured launcher is an existing .exe, then launches it with the asset path. Shell association is also used when no NifSkopec path is configured.

Risk: This is probably acceptable for a user-configured desktop tool, but a tampered config file could cause a user action to launch an unexpected executable. The risk depends on file permissions and whether config can be modified by lower-trust actors.

Evidence: ExternalAssetPathPolicy accepts any rooted existing .exe. ExternalAssetOpenService uses ArgumentList for NifSkopec, which avoids command injection, but does not verify identity of the executable.

Recommendation: Keep the current argument handling, but consider warning on first use, storing only user-selected paths, checking filename/product metadata, or requiring explicit confirmation when the configured executable path changes.

L2. UI startup runs schema initialization synchronously before showing the main window

Severity	Low	Category	Performance / UX
Location	CreationsForge/App.cs:44-58	Validation	Manual test with a locked or slow database; add a startup service test if initialization orchestration is extracted.

Problem: OnFrameworkInitializationCompleted starts diagnostics and calls IDatabaseSchemaInitializer.Initialize before resolving MainWindow.

Risk: If migrations become expensive or the database is locked, startup can appear frozen before the main window/progress UI exists.

Evidence: The call is currently small in validation, but it is on the startup path before desktop.MainWindow is set.

Recommendation: Keep simple migrations synchronous if acceptable, or move initialization behind a startup/progress surface with cancellation and user-facing failure details when migration cost grows.

L3. Unsafe blocks are enabled for the whole UI project

Severity	Low	Category	Best Practice
Location	CreationsForge/CreationsForge.csproj: 16	Validation	Build after moving unsafe code or tightening project settings; search for unsafe usage before and after.

Problem: AllowUnsafeBlocks is enabled at project scope.

Risk: Silk.NET/OpenGL integrations may need unsafe code, but project-wide enablement increases the area where future unsafe code can be added without friction.

Evidence: CreationsForge.csproj line 16 sets AllowUnsafeBlocks true.

Recommendation: If possible, isolate unsafe code to a small rendering/interop project or document the exact files requiring unsafe blocks. If not feasible, keep analyzer review around unsafe usage tight.

Confirmed Strengths

- No hardcoded secrets or credentials were found by targeted repository search.
- dotnet list package --vulnerable --include-transitive reported no vulnerable packages from NuGet advisory data.
- Runtime SQL reviewed in the inspected repositories uses named parameters; dynamic table names in typed record repository code are allowlisted.
- SQLite connections enable foreign keys and use a connection-string builder for database paths.
- Database reset validates the target filename and either default application-data location or recognizable CreationsForge database marker before deletion.
- Asset loose-file resolution prevents data-folder traversal by comparing full paths, and external process launching uses ProcessStartInfo.ArgumentList for the NifSkopec asset argument.
- BA2/BSA readers use checked arithmetic, range checks, decompression output checks, and a 128 MB preview asset byte limit.
- Test coverage includes archive parsing, NIF preview parsing, database reset, import workflows, record comparison, and headless presentation paths.

Areas Needing Human Review

- Application-data, database, and log directory ACLs/permissions on installed machines.
- Actual production/support process for sharing logs, screenshots, and SQLite databases containing ImportDetails.
- CI configuration, if any, was not visible in the inspected repository file list.
- Real-world archive sizes, plugin counts, and import timing on target machines.
- Whether NifSkopec executable path is only user-selected and protected from lower-trust modification.

Prioritized Remediation Plan

Order	Fix	Effort	Why first
1	Fix partial-import stale-row behavior and add regression tests.	M	Protects data correctness and user trust in comparisons.
2	Limit plugin search in SQL and add exact lookup path.	S	Low-risk performance win for an interactive UI path.
3	Add shared analyzer/warnings-as-errors build policy.	S	Prevents quality regressions and aligns code with repo rules.

Order	Fix	Effort	Why first
4	Dispose cancellation sources and formalize VM disposal ownership.	S-M	Small reliability cleanup before more background workflows are added.
5	Sanitize persisted/displayed diagnostics and define support-log redaction expectations.	M	Reduces accidental local-environment disclosure in reports/log bundles.

Validation Commands Run

- dotnet restore ./CreationsForge.sln - passed.
- dotnet build ./CreationsForge.sln --no-restore - passed, 0 warnings, 0 errors.
- dotnet test ./CreationsForge.UnitTests/CreationsForge.UnitTests.csproj --no-build - passed, 364 tests.
- dotnet test ./CreationsForge.PresentationTests/CreationsForge.PresentationTests.csproj --no-build - passed, 50 tests.
- dotnet list ./CreationsForge.sln package --vulnerable --include-transitive - initial sandbox/network attempt failed with NU1301; rerun with NuGet network approval passed and reported no vulnerable packages.